

Activation Functions and Loss Functions for neural networks — How to pick the right one?

When one starts to develop their own neural networks, it is easy to get overwhelmed by the wide variety of options available for each parameter of the model. Which activation function to use for each hidden layer? Which activation function for the output layer? When to use Binary Cross Entropy vs Categorical Cross Entropy?

Such questions will keep coming till we do not have a firm understanding of what each option does, it's pros and cons and when should one use it. The purpose of the blog is exactly that. We will be going through the key features of popular Activation Functions and Loss Functions as well as understand when should one use which. In case you need a refresher on how neural networks work or what is a activation or loss function, please refer to this [blog](#). So without any further delay let's dive in!

Activation Functions

The activation function of a neuron defines it's output given its inputs. We will be talking about 4 popular activation functions:

1. Sigmoid Function:

Description: Takes a real-valued number and scales it between 0 and 1. Large negative numbers become 0 and large positive numbers become 1

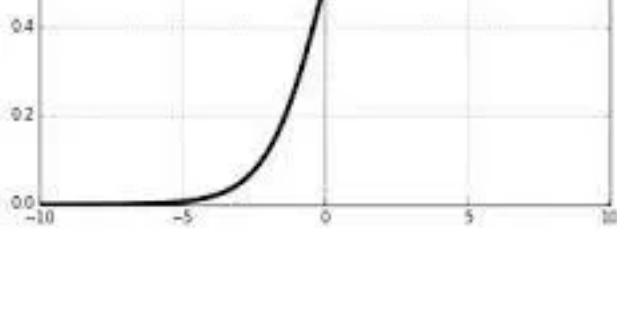
Formula: $1 / (1 + e^{-x})$

Range: (0,1)

Pros: As it's range is between 0 and 1, it is ideal for situations where we need to predict the probability of an event as an output.

Cons: The gradient values are significant for range -3 and 3 but become much closer to zero beyond this range which almost kills the impact of the neuron on the final output. Also, sigmoid outputs are not zero-centered (it is centred around 0.5) which leads to undesirable zig-zagging dynamics in the gradient updates for the weights

Plot:



2. Tanh Function:

Description: Similar to sigmoid but takes a real-valued number and scales it between -1 and 1. It is better than sigmoid as it is centred around 0 which leads to better convergence

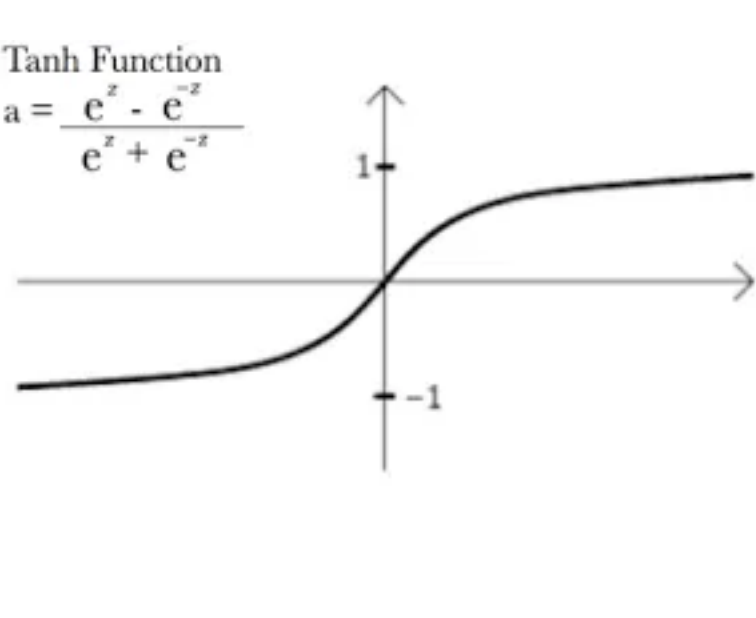
Formula: $(e^x - e^{-x}) / (e^x + e^{-x})$

Range: (-1,1)

Pros: The derivatives of the tanh are larger than the derivatives of the sigmoid which help us minimize the cost function faster

Cons: Similar to sigmoid, the gradient values become close to zero for wide range of values (this is known as vanishing gradient problem). Thus, the network refuses to learn or keeps learning at a very small rate.

Plot:



3. Softmax Function:

Description: Softmax function can be imagined as a combination of multiple sigmoids which can return the probability for a datapoint belonging to each individual class in a multiclass classification problem

Formula:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax
 $\vec{z}$ = input vector
 e^x = standard exponential function for input vector
 K = number of classes in the multi-class classifier
 e^{z_j} = standard exponential function for output vector

Range: (0,1), sum of output = 1

Pros: Can handle multiple classes and give the probability of belonging to each class

Cons: Should not be used in hidden layers as we want the neurons to be independent. If we apply it then they will be linearly dependent.

Plot: Not Applicable

4. ReLU Function:

Description: The rectified linear activation function or ReLU for short is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero. This is the default function but modifying default parameters allows us to use non-zero thresholds and to use a non-zero multiple of the input for values below the threshold (called Leaky ReLU).

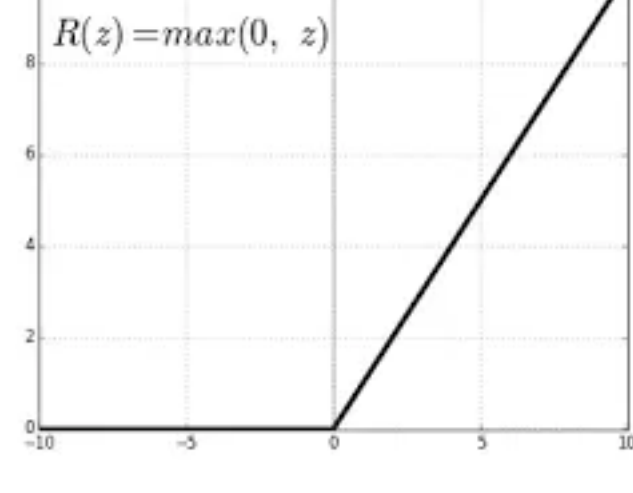
Formula: $\max(0, x)$

Range: (0,inf)

Pros: Although ReLU looks and acts like a linear function, it is a nonlinear function allowing complex relationships to be learned and is able to allow learning through all the hidden layers in a deep network by having large derivatives.

Cons: It should not be used as the final output layer for either classification/regression tasks

Plot:



Loss Functions

The other key aspect in setting up the neural network infrastructure is selecting the right loss functions. With neural networks, we seek to minimize the error (difference between actual and predicted value) which is calculated by the loss function. We will be discussing 3 popular loss functions:

1. Mean Squared Error, L2 Loss

Description: MSE loss is used for regression tasks. As the name suggests, this loss is calculated by taking the mean of squared differences between actual(target) and predicted values.

Formula:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error
 n = number of data points
 Y_i = observed values
 $\hat{Y}_i$ = predicted values

Range: (0,inf)

Pros: Preferred loss function if the distribution of the target variable is Gaussian as it has good derivatives and helps the model converge quickly

Cons: Is not robust to outliers in the data (unlike loss functions like Mean Absolute Error) and penalizes high and low predictions exponentially (unlike loss functions like Mean Squared Logarithmic Error Loss)

2. Binary Cross Entropy

Description: BCE loss is the default loss function used for the binary classification tasks. It requires one output layer to classify the data into two classes and the range of output is (0-1) i.e. should use the sigmoid function.

Formula:

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

where y is the actual label, $\hat{y}$ is the classifier's predicted probability distributions for predicting one class and m is the number of records.

Range: (0,inf)

Pros: The continuous nature of the loss function helps the training process converged well

Cons: Can only be used with sigmoid activation function. Other loss functions like Hinge or Squared Hinge Loss can work with tanh activation function

3. Categorical Cross Entropy

Description: It is the default loss function when we have a multi-class classification task. It requires the same number of output nodes as the classes with the final layer going through a softmax activation so that each output node has a probability value between (0-1).

Formula:

$$logloss = -\frac{1}{N} \sum_i^N \sum_j^M y_{ij} \log(p_{ij})$$

- N is the number of rows
- M is the number of classes

where y is the actual label and p is the classifier's predicted probability distributions for predicting the class j

Range: (0,inf)

Pros: Similar to Binary Cross Entropy, the continuous nature of the loss function helps the training process converged well

Cons: May require a one-hot encoded vector with many zero values if there are many classes, requiring significant memory (should use Sparse Categorical Crossentropy in this case)

Summary

After reading this blog, the readers should be able to set up the correct architecture (in terms of activation and loss functions) for most of the popular machine learning problems. The table below gives a snapshot of where to use each of these functions:

Problem Type	Last-layer Output Nodes	Hidden-layer activation	Last-layer activation	Loss function
Binary classification	1	ReLU (first choice), Tanh (for RNNs)	Sigmoid	Binary Crossentropy
Multi-class, single-label classification	Number of classes		Softmax	Categorical Crossentropy
Multi-class, multi-label classification	Number of classes		Sigmoid (one for each class)	Binary Crossentropy
Regression to arbitrary values	1		None	MSE
Regression to values between 0 and 1	1		Sigmoid	MSE/Binary Crossentropy

There are a few other activation functions like softplus and elu as well as other loss functions like hinge and huber available in the deep learning packages/libraries. I would definitely encourage interested readers to go through these other functions and try them out (specially, if the ones discussed here do not yield the right results).

The next step for building robust and accurate neural networks is understanding the various problems one might face while training neural networks and how to deal with them. Please do read the [3rd part](#) of this series to learn more on this.